

# **Architectures de l'Intelligence : Une Synthèse de l'Ingénierie de Prompt Moderne, de la Génération Augmentée par Récupération et des Techniques de Raisonnement Avancées des LLM**

## **Partie I : Les Fondements de l'Interaction avec les Modèles - Ingénierie de Prompt et de Contexte Avancée**

Cette section fondamentale déconstruit les principes de la communication efficace avec les Grands Modèles de Langage (LLM). Elle dépasse les simples « trucs et astuces » pour établir un cadre systématique de conception d'instructions, traçant une ligne évolutive claire de l'ingénierie de prompt de base au concept plus holistique d'« ingénierie de contexte ».

### **1.1 Un Cadre Unifié des Principes de l'Ingénierie de Prompt**

L'analyse des guides de bonnes pratiques des principaux fournisseurs de modèles — OpenAI, Anthropic et Google — révèle un ensemble de stratégies fondamentales et convergentes. Ces principes constituent le socle de toute interaction efficace et prévisible avec les LLM.

#### **La Clarté et la Spécificité comme Mandat Principal**

Le principe le plus universel est l'exigence de précision. Des instructions vagues ou ambiguës conduisent inévitablement à des réponses génériques ou incorrectes.<sup>1</sup> Les guides officiels soulignent unanimement que la qualité de la sortie est directement proportionnelle à la clarté de l'entrée. Le guide d'OpenAI insiste sur la nécessité d'être « spécifique, descriptif et aussi détaillé que possible » concernant le contexte, le résultat, la longueur et le format souhaités.<sup>2</sup> Il recommande également de réduire les descriptions « vagues et imprécises » au profit d'instructions concises.<sup>2</sup> De même, la documentation de Google Cloud préconise de « fixer des buts et des objectifs clairs » en utilisant des verbes d'action pour spécifier l'action désirée, comme « Rédiger une liste à puces qui résume... ».<sup>3</sup> Cette insistance sur la précision n'est pas une simple suggestion stylistique ; elle est essentielle pour contraindre l'espace de recherche du modèle et le guider vers la sortie la plus probable et la plus pertinente.

## **Le Pouvoir des Exemples (Few-Shot Prompting)**

Une autre technique universellement recommandée est l'apprentissage par l'exemple, connu sous le nom de *few-shot prompting*. Le fait de montrer au modèle le comportement attendu est souvent plus efficace que de le décrire textuellement. OpenAI détaille explicitement les approches *zero-shot* (sans exemple) et *few-shot* (avec quelques exemples), positionnant cette dernière comme une étape clé lorsque les instructions simples échouent.<sup>2</sup> Anthropic intègre les exemples comme un composant critique de sa structure de prompt recommandée, suggérant qu'un ou deux exemples de ce qui constitue un bon résultat améliorent considérablement la cohérence.<sup>4</sup> Google confirme que « des exemples de haute qualité aident à définir les bonnes attentes en matière de format et de ton ».<sup>5</sup> Cette technique exploite la capacité fondamentale des LLM à reconnaître et à reproduire des motifs, ce qui en fait un outil puissant pour spécifier des formats de sortie complexes ou des styles nuancés.

## **Structurer le Prompt pour une Lisibilité par la Machine**

La manière dont un prompt est structuré influence directement la façon dont un modèle alloue son attention et interprète les instructions. L'utilisation de délimiteurs clairs pour séparer les différentes sections d'un prompt est une pratique courante et efficace. OpenAI recommande d'utiliser des marqueurs comme ### ou des triples guillemets "" pour isoler les instructions du contexte textuel.<sup>2</sup> Anthropic va plus loin en préconisant fortement l'utilisation de balises de type XML (par exemple, <instructions>, <contexte>) pour délimiter les sections de manière sémantique.<sup>6</sup> Cette approche ne vise pas seulement à améliorer la clarté pour un lecteur humain, mais aussi à fournir une structure explicite que le modèle peut parser, lui

permettant de distinguer sans ambiguïté les instructions, les données d'entrée, les exemples et les règles.

## **Amorçage Instructionnel et Cadrage Positif**

La position des instructions dans le prompt a une importance significative. OpenAI recommande de placer les instructions au début du prompt pour les « amorcer » dans l'attention du modèle.<sup>2</sup> De plus, une technique subtile mais puissante, mise en avant par OpenAI et Google, consiste à formuler les instructions de manière positive : dire au modèle *ce qu'il faut faire* plutôt que *ce qu'il ne faut pas faire*.<sup>2</sup> Par exemple, au lieu de dire « NE PAS demander le nom d'utilisateur », il est plus efficace de formuler : « L'agent s'abstiendra de poser des questions relatives aux informations personnelles identifiables (PII) et orientera l'utilisateur vers l'article d'aide ». Cette approche constructive évite de focaliser l'attention du modèle sur le comportement négatif et le guide plus directement vers l'action souhaitée.

## **1.2 Philosophies Divergentes dans l'Architecture de Prompt**

Au-delà des principes fondamentaux communs, les principaux laboratoires de recherche en IA ont développé des philosophies distinctes en matière d'architecture de prompt. Ces approches « opinionnées » révèlent des hypothèses sous-jacentes différentes sur la manière dont les modèles traitent l'information et raisonnent. Comprendre ces divergences permet aux praticiens d'adapter leur stratégie au modèle spécifique qu'ils utilisent.

### **Le Modèle de Contexte Structuré d'Anthropic**

Anthropic promeut une approche de l'ingénierie de prompt qui est hautement structurée, quasi-programmatique. Leur guide interne, révélé publiquement, détaille un processus en dix étapes qui organise le contexte de manière hiérarchique.<sup>4</sup> Cette structure superpose les informations, allant du général au spécifique :

1. **Contexte de la Tâche (Persona)** : Définir clairement qui est l'IA et quel rôle elle joue.
2. **Contexte du Ton** : Spécifier le ton exact (par exemple, « professionnel mais accessible »).
3. **Données/Documents de Fond** : Fournir tous les documents pertinents (rapports,

e-mails, guides de style).

4. **Description Détaillée de la Tâche et Règles** : Établir des limites et des contraintes claires (par exemple, « ne jamais dépasser 500 mots »).
5. **Exemples** : Montrer un ou deux exemples de la sortie souhaitée.
6. **Historique de la Conversation** : Inclure les échanges précédents pertinents.
7. **Description de la Tâche Immédiate** : Énoncer clairement la demande actuelle.
8. **Étape de Réflexion** : Ajouter une instruction comme « Pensez à votre réponse avant de répondre ».
9. **Formatage de la Sortie** : Spécifier la structure exacte en utilisant des balises XML ou du Markdown.
10. **Réponse Pré-remplie (Avancé)** : Commencer la réponse pour guider le style du modèle.

Cette méthodologie est formalisée dans leur concept d'« ingénierie de contexte », qui considère l'ensemble de la fenêtre de contexte comme un état programmable à optimiser.<sup>6</sup> L'utilisation de balises XML est un détail d'implémentation clé, suggérant que le modèle est perçu comme un processeur de données structurées qui bénéficie de frontières sémantiques claires.

## L'Approche Itérative et Instructionnelle d'OpenAI

La documentation d'OpenAI, bien que valorisant également la structure, met l'accent sur un processus de développement itératif. Leurs meilleures pratiques encouragent à commencer simplement (zero-shot) et à ajouter progressivement de la complexité (few-shot, puis fine-tuning) en fonction des échecs observés.<sup>2</sup> Cette approche est plus proche d'un cycle de débogage et de raffinement. OpenAI se concentre également sur des instructions tactiques telles que « décomposer la tâche en sous-tâches plus simples » et « amorcer la sortie » en commençant la réponse du modèle pour le guider dans la bonne direction.<sup>7</sup> Cette philosophie traite le modèle moins comme un système rigide et plus comme un collaborateur intelligent qui nécessite des instructions claires et un feedback correctif pour s'améliorer.

## L'Accent de Google sur la Clarté et la Réutilisabilité

Les meilleures pratiques de Google, telles que résumées dans leurs guides, mettent l'accent sur la clarté, la simplicité et la réutilisabilité.<sup>3</sup> Ils préconisent des prompts clairs, concis et axés sur des verbes d'action. Une contribution notable est l'encouragement à utiliser des variables ou des espaces réservés (placeholders) dans les prompts, ce qui permet de créer des

modèles réutilisables et dynamiques.<sup>5</sup> De plus, Google insiste fortement sur la spécification de formats de sortie structurés comme JSON ou CSV, reconnaissant que cela facilite grandement le traitement programmatique des réponses du modèle et réduit le besoin de nettoyage post-génération.<sup>5</sup>

La juxtaposition de ces approches révèle des modèles mentaux implicites distincts. L'approche d'Anthropic, avec ses balises XML et sa hiérarchie stricte, traite le LLM comme un système nécessitant un appel d'API bien formé. L'approche d'OpenAI, itérative et axée sur les instructions, le traite comme un étudiant intelligent mais parfois naïf. L'approche de Google, axée sur les modèles et les sorties structurées, le positionne comme un composant de service dans une architecture logicielle plus large. Un praticien avancé ne devrait pas suivre aveuglément un seul guide, mais plutôt comprendre ces philosophies sous-jacentes pour adapter son style de prompt à l'architecture du modèle et à la nature de la tâche.

## **1.3 De l'Ingénierie de Prompt à l'Ingénierie de Contexte**

Le domaine lui-même évolue. Le terme « ingénierie de prompt » devient de plus en plus insuffisant pour décrire la complexité du travail requis pour construire des applications LLM robustes.

### **Le Changement de Paradigme**

Comme le souligne Anthropic, le travail consiste moins à trouver les bons mots et plus à déterminer « quelle configuration de contexte est la plus susceptible de générer le comportement souhaité de notre modèle ». <sup>6</sup> Cela marque un passage fondamental de la rédaction d'une chaîne de caractères impérative à l'architecture d'un environnement informationnel complet pour le LLM. Cette nouvelle discipline, baptisée « ingénierie de contexte », englobe une vision plus holistique.

### **Le Contexte comme État Programmable**

Dans ce nouveau paradigme, la fenêtre de contexte est considérée comme un état holistique et programmable. Cet état inclut non seulement le prompt système, mais aussi les documents récupérés (dans un système RAG), les définitions d'outils (pour l'appel de fonctions) et

l'historique de la conversation. La gestion efficace de cet état devient un problème d'ingénierie à part entière. Des techniques comme la **compaction** (résumer ou compresser l'historique de la conversation pour économiser des jetons) et la **prise de notes structurée** (demander au modèle de maintenir un « carnet de notes » interne dans le contexte pour suivre les informations clés sur de longues interactions) sont des stratégies émergentes pour gérer cet état de manière efficace.<sup>6</sup>

Cette évolution reflète une convergence avec les principes classiques du génie logiciel. Des concepts tels que l'utilisation de variables, la définition de schémas de sortie (similaires aux signatures de fonction et aux types de retour), et la décomposition de tâches complexes en sous-tâches plus simples<sup>1</sup> sont des parallèles directs. L'avenir de l'interaction avec les LLM ne réside pas dans la découverte de « mots magiques », mais dans l'application d'une discipline d'ingénierie rigoureuse au processus d'instruction des modèles. Les compétences en conception modulaire, en gestion d'état et en définition d'API claires (pour les prompts et les outils) deviendront plus précieuses que la capacité à rédiger des phrases en langage naturel astucieuses.

| Principe                      | Approche d'OpenAI                                                                                                                      | Approche d'Anthropic                                                                                                      | Approche de Google                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Spécificité</b>            | Soyez aussi détaillé que possible sur le contexte, le résultat, la longueur, le format, le style, etc.. <sup>2</sup>                   | Définissez clairement le rôle (persona), le ton et les règles. La spécificité est essentielle. <sup>4</sup>               | Utilisez des verbes d'action et définissez la longueur, le format et le public cible. <sup>3</sup>  |
| <b>Structure</b>              | Placez les instructions au début et utilisez des délimiteurs comme ### ou """" pour séparer les instructions du contexte. <sup>2</sup> | Utilisez des balises XML (par exemple, <instructions>) pour délimiter sémantiquement les sections du prompt. <sup>6</sup> | Préférez les formats de sortie structurés comme JSON ou CSV pour faciliter le parsing. <sup>5</sup> |
| <b>Utilisation d'Exemples</b> | Commencez par le <i>zero-shot</i> , puis passez au <i>few-shot</i> en fournissant des paires d'exemples                                | Incluez 1 à 2 exemples de ce qui constitue un bon résultat pour améliorer                                                 | Des exemples de haute qualité aident à définir les bonnes attentes en matière de format et de       |

|                   |                                                                                                     |                                                                                                                        |                                                                                                                                                              |
|-------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                   | entrée/sortie. <sup>2</sup>                                                                         | considérablement la cohérence. <sup>4</sup>                                                                            | ton. <sup>5</sup>                                                                                                                                            |
| <b>Persona</b>    | Implicite dans les instructions, mais pas un élément de premier niveau.                             | Le premier et le plus important élément du prompt est de définir QUI est l'IA et quel rôle elle joue. <sup>4</sup>     | Spécifiez le rôle pour aider le modèle à adopter le ton et la perspective corrects (par exemple, « Agissez en tant que conseiller financier »). <sup>1</sup> |
| <b>Cadrage</b>    | Dites ce qu'il faut faire plutôt que ce qu'il ne faut pas faire. <sup>2</sup>                       | L'accent est mis sur la définition de règles et de contraintes positives. <sup>4</sup>                                 | Préférez les instructions positives, sauf pour les règles de sécurité strictes. <sup>5</sup>                                                                 |
| <b>Complexité</b> | Décomposez les tâches complexes en une série de prompts plus simples et plus gérables. <sup>7</sup> | La structure hiérarchique est conçue pour gérer des tâches complexes en un seul prompt bien architecturé. <sup>4</sup> | Décomposez les requêtes complexes en étapes plus petites pour éviter la confusion du modèle. <sup>1</sup>                                                    |

## Partie II : Augmentation des Connaissances

### Paramétriques - Architecture des Systèmes RAG Modernes

Cette section passe de l'instruction du modèle à la fourniture de connaissances externes. Elle propose un guide complet pour la construction de systèmes de Génération Augmentée par Récupération (RAG), en partant du pipeline fondamental et en ajoutant systématiquement des techniques avancées pour surmonter les modes d'échec courants.

## 2.1 Le Pipeline RAG Canonique : Un Guide d'Implémentation Pratique

Le RAG est une technique puissante qui améliore les LLM en les combinant avec des bases de connaissances externes, comblant ainsi le fossé entre les connaissances statiques du modèle et les informations dynamiques et spécifiques à un domaine.<sup>9</sup> Un pipeline RAG de base, souvent appelé *naive RAG*, suit un processus simple mais efficace en plusieurs étapes.<sup>10</sup>

### Composants Fondamentaux

Le processus RAG peut être décomposé en cinq étapes fondamentales : le chargement, la division, l'intégration, le stockage et la récupération/génération.<sup>9</sup>

1. **Chargement (Loading)** : La première étape consiste à ingérer les données à partir de leur source d'origine. Des frameworks comme LangChain fournissent une multitude de chargeurs de documents (Document Loaders) pour gérer divers formats, tels que PyPDFLoader pour les fichiers PDF ou WebBaseLoader pour extraire le contenu de pages web.<sup>11</sup>
2. **Division (Splitting)** : Les documents bruts sont souvent trop volumineux pour être traités directement par un LLM. Ils doivent être divisés en morceaux plus petits et gérables, appelés *chunks*. Le RecursiveCharacterTextSplitter est un outil polyvalent qui divise le texte en fonction d'une liste de séparateurs (par exemple, paragraphes, phrases) tout en maintenant la cohérence sémantique.<sup>11</sup> Le réglage de la taille des morceaux (*chunk\_size*) et du chevauchement (*chunk\_overlap*) est un aspect crucial de l'optimisation du RAG.
3. **Intégration (Embedding)** : Chaque morceau de texte est ensuite transformé en une représentation numérique, ou vecteur, à l'aide d'un modèle d'intégration (*embedding model*). Des modèles comme text-embedding-ada-002 d'OpenAI<sup>11</sup> ou des modèles open-source de Hugging Face capturent la signification sémantique du texte. Ces vecteurs permettent de comparer la signification des morceaux de texte entre eux et avec la requête de l'utilisateur.
4. **Stockage (Storing)** : Les vecteurs d'intégration sont stockés et indexés dans une base de données vectorielle (*vector store*). Pour le développement local, des options légères comme Chroma<sup>11</sup> ou FAISS<sup>12</sup> sont populaires. Pour des applications à plus grande échelle, des bases de données comme Milvus<sup>13</sup> ou Pinecone offrent de meilleures performances et une meilleure scalabilité.
5. **Récupération et Génération (Retrieval & Generation)** : Lorsqu'un utilisateur pose une



question, celle-ci est également convertie en un vecteur d'intégration. Le système effectue alors une recherche de similarité (généralement par le plus proche voisin) dans la base de données vectorielle pour trouver les morceaux de texte dont les vecteurs sont les plus proches de celui de la question.<sup>11</sup> Ces morceaux pertinents sont récupérés et insérés dans un modèle de prompt (prompt template) avec la question originale. Ce contexte enrichi est ensuite fourni au LLM, qui génère une réponse finale fondée sur les informations récupérées.<sup>9</sup>

Le framework LlamaIndex est réputé pour sa simplicité, illustrée par son célèbre « démarrage en 5 lignes » qui encapsule ce processus de manière concise<sup>14</sup> :

Python

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()
response = query_engine.query("Posez une question sur les données ici")
```

## 2.2 RAG Avancé : Une Taxonomie des Techniques d'Amélioration

Le pipeline RAG de base, bien qu'efficace, présente des limites. Les systèmes RAG avancés introduisent une série de techniques pour améliorer la qualité et la pertinence de chaque étape du processus. Ces techniques peuvent être classées en trois catégories : l'optimisation pré-récupération, l'optimisation de la récupération et l'optimisation post-récupération.

### Optimisation Pré-Récupération (Améliorer le "R" de RAG)

Ces techniques se concentrent sur l'amélioration de la qualité des données avant même que la recherche ne soit effectuée.

- **Stratégies de Découpage Sophistiquées** : Au lieu d'un découpage à taille fixe, le **découpage sémantique** divise le texte en fonction de la signification, par exemple aux limites des paragraphes ou des sections, garantissant que chaque morceau représente

une idée autonome.<sup>10</sup> Le **découpage à taille dynamique** ajuste la taille des morceaux en fonction du type de contenu (par exemple, des morceaux plus petits pour des manuels techniques denses, des morceaux plus grands pour des rapports narratifs).<sup>10</sup>

- **Filtrage par Métadonnées** : L'une des améliorations les plus impactantes consiste à enrichir les documents avec des métadonnées (par exemple, source, date, catégorie, auteur).<sup>10</sup> Le système de récupération peut alors utiliser ces balises pour pré-filtrer l'espace de recherche. Par exemple, si un utilisateur demande des informations sur la « politique de sécurité de 2024 », le système peut filtrer pour ne rechercher que dans les documents de type « politique » datés de « 2024 », ce qui améliore considérablement la pertinence des résultats.<sup>15</sup>
- **Indexation Hiérarchique** : Des techniques comme la **récupération Parent-Enfant** (*Parent-Child Retrieval*) abordent le compromis entre la précision de la récupération et la richesse du contexte. Des morceaux plus petits et plus précis (enfants) sont utilisés pour l'intégration et la recherche, ce qui permet une meilleure correspondance sémantique. Cependant, une fois le morceau enfant pertinent trouvé, c'est le morceau parent plus grand et plus contextuel qui est fourni au LLM pour la génération de la réponse.<sup>15</sup>

## Optimisation de la Récupération (Améliorer la Recherche)

Ces techniques visent à améliorer le processus de recherche lui-même.

- **Recherche Hybride** : Cette approche combine la recherche sémantique (vectorielle) avec la recherche par mots-clés traditionnelle (comme l'algorithme BM25).<sup>15</sup> La recherche sémantique excelle à trouver des contenus conceptuellement similaires, tandis que la recherche par mots-clés garantit une correspondance exacte pour les termes spécifiques, les noms de produits ou les acronymes.
- **Transformations de Requête** : Au lieu d'utiliser la requête brute de l'utilisateur, ces méthodes la modifient pour la rendre plus efficace pour la récupération. La **génération de requêtes multiples** (*Multi-Query Generation*) utilise un LLM pour créer plusieurs variantes de la question originale, élargissant ainsi la portée de la recherche.<sup>10</sup> Les **intégrations de documents hypothétiques** (*Hypothetical Document Embeddings* ou HyDE) poussent ce concept plus loin : un LLM génère d'abord une réponse idéale hypothétique à la question de l'utilisateur. C'est l'intégration de cette *réponse inventée* qui est ensuite utilisée pour rechercher des documents réels, ce qui permet de combler le fossé sémantique entre une question concise et des documents détaillés.<sup>15</sup>

## Optimisation Post-Récupération (Améliorer le "A" et le "G" de RAG)

Ces techniques affinent les résultats après la récupération initiale.

- **Re-classement (Re-ranking)** : Après une première récupération rapide (par exemple, en utilisant un bi-encodeur), un modèle de **cross-encoder** plus puissant mais plus lent est utilisé pour réévaluer et re-classer un sous-ensemble plus restreint des documents les mieux classés.<sup>15</sup> Cela permet de pousser les résultats les plus pertinents en haut de la liste, ce qui est crucial pour lutter contre le problème du « lost in the middle », où les LLM accordent moins d'attention au contexte situé au milieu d'un long prompt.<sup>15</sup>
- **Compression du Contexte** : Les documents récupérés peuvent contenir des informations redondantes ou non pertinentes. Les techniques de compression du contexte visent à filtrer ce bruit avant de transmettre les informations au LLM, garantissant que le contexte final est dense, pertinent et efficace en termes de jetons.

La construction d'un système RAG de production nécessite une approche holistique. Le choix d'une stratégie de découpage, par exemple, a des implications directes sur le modèle d'intégration, la méthode de récupération et le modèle de prompt final. Optimiser un composant de manière isolée sans tenir compte de ses effets sur le reste du pipeline est une cause fréquente d'échec. L'ensemble du pipeline doit être co-conçu et évalué comme un système unifié.

## 2.3 L'Écosystème RAG : Frameworks et Outils

Le développement de systèmes RAG est soutenu par un écosystème dynamique de frameworks et d'outils open-source.

- **LangChain** : Un framework polyvalent pour composer des applications LLM, y compris des pipelines RAG complexes. Il offre une vaste bibliothèque de composants intégrés (chargeurs, diviseurs, bases de données vectorielles) qui peuvent être assemblés pour créer des solutions personnalisées.<sup>9</sup> Ses développements récents, comme LangGraph, montrent la voie vers des systèmes RAG plus agentiques et autonomes.<sup>16</sup>
- **LlamaIndex** : Un framework spécifiquement optimisé pour la construction d'applications LLM augmentées par le contexte. Il met un accent particulier sur les pipelines d'ingestion et d'indexation de données, offrant des abstractions de haut niveau pour simplifier la construction de moteurs de recherche et de chatbots.<sup>14</sup> Ses services gérés, comme LlamaParse pour l'analyse de documents complexes, soulignent l'importance croissante de la qualité des données en amont.<sup>14</sup>
- **Hugging Face** : Le centre névralgique de l'écosystème open-source. Il fournit non seulement une vaste collection de modèles pré-entraînés (pour l'intégration et la génération), mais aussi des bibliothèques fondamentales comme transformers et

datasets qui constituent les briques de base de nombreux systèmes RAG.<sup>18</sup> Leurs *cookbooks* et tutoriels offrent des guides pratiques pour l'implémentation et l'évaluation des pipelines RAG.<sup>20</sup>

| Technique                                      | Étape            | Mécanisme                                                                                                                  | Problème Résolu                                                                                              | Compromis                                                                              |
|------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Découpage Sémantique</b>                    | Pré-Récupération | Divise le texte en fonction des changements de sujet ou des limites de paragraphes.                                        | Morceaux de texte coupés au milieu d'une idée, contexte incohérent.                                          | Peut nécessiter des modèles plus complexes pour identifier les frontières sémantiques. |
| <b>Récupération Parent-Enfant</b>              | Pré-Récupération | Intègre de petits morceaux (enfants) pour la recherche, mais récupère des morceaux plus grands (parents) pour le contexte. | Compromis entre la précision de la recherche (petits morceaux) et la richesse du contexte (grands morceaux). | Augmente la complexité de la gestion des données et de l'indexation.                   |
| <b>Recherche Hybride (BM25 + Vecteur)</b>      | Récupération     | Combine la recherche par mots-clés pour la précision et la recherche sémantique pour la pertinence conceptuelle.           | Échec de la récupération pour les acronymes, les identifiants ou les termes spécifiques.                     | Nécessite la maintenance de deux systèmes d'indexation distincts.                      |
| <b>HyDE (Hypothetical Document Embeddings)</b> | Récupération     | Génère une réponse hypothétique à la requête, puis recherche                                                               | Décalage sémantique entre la requête de l'utilisateur                                                        | Ajoute une latence et un coût supplémentaires dus à                                    |

|                                        |                   |                                                                                                  |                                                                                                                   |                                                            |
|----------------------------------------|-------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
|                                        |                   | des documents similaires à cette réponse.                                                        | (courte, vague) et les documents (longs, détaillés).                                                              | l'appel LLM initial.                                       |
| <b>Re-classement par Cross-Encoder</b> | Post-Récupération | Utilise un modèle plus puissant pour réévaluer et re-classer les N premiers documents récupérés. | Documents très pertinents "perdus au milieu" du contexte, récupération de documents superficiellement similaires. | Augmente considérablement la latence et le coût de calcul. |

## Partie III : Susciter le Raisonnement Latent - Intégration du CoT et de l'Instruction Tuning

Cette section relie la fonction de fourniture de données du RAG aux processus cognitifs qui permettent à un LLM de raisonner efficacement *à partir* de ces données. Elle synthétise la recherche académique pour expliquer les fondements théoriques du raisonnement avancé.

### 3.1 Le Prompting par Chaîne de Pensée (Chain-of-Thought - CoT)

Le prompting par Chaîne de Pensée (CoT) est une technique qui a révolutionné la capacité des LLM à résoudre des problèmes complexes. Plutôt que de demander une réponse directe, le CoT incite le modèle à décomposer son raisonnement en étapes intermédiaires, imitant un processus de pensée humain.

#### Le Concept Fondamental

L'idée centrale du CoT est de transformer un problème complexe en une séquence d'étapes plus simples et gérables.<sup>21</sup> En rendant le processus de raisonnement explicite, le modèle est moins susceptible de commettre des erreurs logiques et la sortie finale est souvent plus précise. De plus, cela offre une traçabilité et une interprétabilité précieuses, car les utilisateurs peuvent suivre la logique du modèle pour comprendre comment il est parvenu à sa conclusion.<sup>21</sup> La forme la plus simple de cette technique, connue sous le nom de **Zero-Shot CoT**, consiste à ajouter une simple instruction comme « Réfléchissons étape par étape » (Let's think step by step) à la fin du prompt, ce qui suffit à déclencher ce comportement de raisonnement dans les grands modèles.<sup>21</sup>

## L'Évolution du CoT

La technique a évolué depuis sa création. Initialement, le **Few-Shot CoT** consistait à fournir manuellement quelques exemples de problèmes résolus avec des chaînes de pensée détaillées.<sup>21</sup> Bien que très efficace, cette méthode est coûteuse en termes d'annotation manuelle. Des recherches ultérieures se sont concentrées sur des méthodes de prompting automatique, où le modèle est incité à générer ses propres chaînes de pensée sans exemples pré-définis.<sup>21</sup> Le domaine a également exploré des variantes structurelles plus complexes. Des approches comme l'**Arbre de Pensées** (*Tree of Thoughts* - ToT) ou le **Graphe de Pensées** (*Graph of Thoughts*) permettent au modèle d'explorer plusieurs chemins de raisonnement en parallèle, d'évaluer leur viabilité et de revenir en arrière si nécessaire, créant ainsi un processus de résolution de problèmes plus robuste et délibératif.<sup>23</sup>

## Le Rôle Changeant des Exemples

Une observation critique issue de recherches récentes concerne le rôle des exemples dans le CoT. Alors que les premiers travaux ont démontré que le Few-Shot CoT améliorerait considérablement les performances des modèles<sup>21</sup>, des études plus récentes sur des modèles plus puissants (comme la série Qwen2.5) suggèrent que cette amélioration diminue. Pour ces modèles très performants, le principal avantage des exemples CoT semble être d'aligner le *format de sortie* sur les attentes humaines, plutôt que d'enseigner le processus de raisonnement lui-même.<sup>22</sup> Ces modèles possèdent déjà de solides capacités intrinsèques de raisonnement Zero-Shot CoT. De plus, il a été observé que les modèles puissants ont tendance à ignorer le contenu de raisonnement des exemples, surtout s'ils sont perçus

comme bruyants ou trop simples, se fiant plutôt à leurs propres capacités internes.<sup>22</sup>

## **3.2 L'Ajustement par Instructions (Instruction Tuning / Supervised Fine-Tuning - SFT)**

L'ajustement par instructions, également connu sous le nom de fine-tuning supervisé (SFT), est une étape cruciale qui transforme un LLM de base, entraîné à prédire le mot suivant, en un assistant capable de suivre des instructions humaines de manière utile et sûre.

### **Comblar le Fossé entre Prédiction et Intention**

Le SFT est le processus qui consiste à entraîner davantage un LLM pré-entraîné sur un ensemble de données composé de paires (instruction, sortie).<sup>24</sup> Ce processus comble le fossé fondamental entre l'objectif de prédiction de mot suivant du modèle de base et l'objectif de l'utilisateur, qui est que le modèle adhère à ses instructions.<sup>25</sup> Cette étape d'alignement est ce qui confère aux modèles comme GPT-4 ou Claude leur capacité à dialoguer, à répondre à des questions et à effectuer des tâches de manière contrôlable et prévisible.<sup>24</sup>

### **Méthodologie et Ensembles de Données**

La méthodologie du SFT implique la curation ou la génération d'ensembles de données d'instructions de haute qualité et l'entraînement du modèle de manière supervisée pour minimiser la différence entre ses sorties et les sorties de référence.<sup>24</sup> La qualité et la diversité de l'ensemble de données d'instructions sont des facteurs déterminants pour les performances du modèle final. Des efforts de recherche significatifs sont consacrés à la construction de ces ensembles de données, qui peuvent couvrir un large éventail de tâches, de domaines et de modalités.

### **Ajustement Fédéré et Efficace en Données**

À mesure que le SFT devient une pratique courante, des techniques plus avancées émergent. L'**Ajustement par Instructions Fédéré et Efficace en Données** (*Federated Data-Efficient Instruction Tuning*, ou FedHDS) est une approche de pointe qui vise à exploiter de vastes ensembles de données d'instructions privées réparties sur plusieurs clients (par exemple, des appareils d'utilisateurs) sans partager les données brutes, préservant ainsi la confidentialité.<sup>27</sup> Cette méthode sélectionne un sous-ensemble représentatif de données pour réduire la redondance et les coûts de calcul, tout en améliorant la diversité des données et les performances du modèle sur des tâches inédites.<sup>27</sup>

La relation entre ces techniques est symbiotique. Le RAG fournit les connaissances factuelles externes (le « quoi »). Le CoT fournit le processus de raisonnement structuré pour travailler avec ces connaissances, en particulier pour les problèmes en plusieurs étapes (le « comment »). L'ajustement par instructions est le processus d'entraînement fondamental qui rend le modèle capable de comprendre l'instruction d'utiliser le contexte RAG et de suivre le processus CoT en premier lieu (la « capacité »). Un système de pointe doit intégrer les trois : le prompt doit instruire le modèle d'utiliser le contexte RAG fourni et de réfléchir étape par étape pour formuler sa réponse, et la capacité du modèle à le faire de manière fiable est un résultat direct de son ajustement par instructions.

De plus, l'efficacité des techniques de prompting comme le CoT dépend fortement du « pré-calcul » effectué lors de l'entraînement du modèle. Le prompting n'est pas magique ; c'est un moyen d'activer des capacités latentes. Le fait que l'instruction « Réfléchissons étape par étape » fonctionne pour les grands modèles mais pas pour les plus petits indique qu'il s'agit d'une « capacité émergente ».<sup>21</sup> La diminution du bénéfice des exemples *few-shot* à mesure que le modèle de base devient plus puissant <sup>22</sup> suggère que la capacité à effectuer un CoT est « intégrée » lors des phases d'entraînement et d'ajustement. Le prompt *zero-shot* n'est qu'un déclencheur pour cette capacité préexistante. Cela a des conséquences profondes : à mesure que les modèles de base continueront de s'améliorer, l'ingénierie de prompt s'éloignera des structures complexes et manuelles pour se concentrer sur une définition plus claire des objectifs et la fourniture de données de meilleure qualité via le RAG. L'« ingénierie » se déplace du prompt vers le pipeline de données et l'architecture globale du système.

## **Partie IV : L'Émergence des Systèmes Autonomes - RAG Agentique et Appel de Fonctions**

Cette section explore la frontière où les systèmes RAG évoluent de simples répondeurs passifs à des agents proactifs, utilisant des outils et capables d'exécuter des tâches complexes en plusieurs étapes.



## 4.1 RAG Agentique : Le Travailleur du Savoir Auto-Correcteur

Le RAG agentique représente un changement de paradigme par rapport au pipeline RAG linéaire et statique. Il intègre des agents autonomes capables de gérer dynamiquement les stratégies de récupération, d'évaluer la qualité des informations et d'affiner itérativement leur compréhension pour accomplir des tâches complexes.<sup>28</sup>

### Au-delà de la Simple Récupération

Contrairement au RAG traditionnel qui suit une séquence fixe (récupérer, puis générer), un système RAG agentique introduit une boucle de raisonnement et d'action. Ces systèmes ne se contentent pas de récupérer des informations ; ils peuvent évaluer la pertinence de ces informations, décider si elles sont suffisantes et, si ce n'est pas le cas, prendre des mesures correctives.<sup>16</sup>

### Modèles Agentiques Fondamentaux

Cette autonomie est rendue possible par l'implémentation de modèles de conception agentiques clés :

- **Réflexion** : La capacité de l'agent à évaluer de manière critique son propre travail, les informations qu'il a récupérées ou les réponses qu'il a générées.
- **Planification** : La capacité de décomposer une tâche complexe en une séquence d'étapes ou de sous-tâches.
- **Utilisation d'Outils** : La capacité de décider quel outil (y compris le récupérateur RAG lui-même) utiliser, quand l'utiliser et avec quels paramètres.<sup>28</sup>

### Implémentation Pratique avec LangGraph

Le framework LangGraph de LangChain fournit une excellente illustration pratique de la construction d'un RAG agentique.<sup>16</sup> Au lieu d'un pipeline linéaire, le système est modélisé

comme un graphe d'états cyclique. Ce graphe permet à l'agent d'exécuter une boucle de raisonnement qui peut inclure les étapes suivantes :

1. **Évaluation des Documents** : Après une récupération initiale, un nœud du graphe évalue si les documents récupérés sont pertinents pour la question. Cette évaluation peut être effectuée par un appel LLM.
2. **Réécriture de la Question** : Si les documents sont jugés non pertinents, l'agent conclut que la question initiale était peut-être mal formulée pour la récupération. Un autre nœud du graphe est alors activé pour réécrire la question afin de la rendre plus efficace, puis le processus de récupération est relancé.
3. **Génération de la Réponse** : L'agent ne procède à la génération de la réponse finale que lorsqu'il a déterminé, via la boucle d'évaluation, qu'il dispose de documents de haute confiance.

Cette boucle de récupération, d'évaluation et de réécriture est l'essence d'un processus agentique. Elle transforme le RAG d'un processus de récupération de données en une méthode quasi-scientifique : l'agent formule une hypothèse (la requête), mène une expérience (la récupération), observe les résultats (les documents), évalue leur qualité, puis affine son hypothèse (réécrit la requête) si les résultats sont insatisfaisants. Il s'agit d'une architecture beaucoup plus robuste et fiable que le RAG naïf, car elle intègre un mécanisme d'auto-correction capable de gérer l'ambiguïté et les échecs de récupération, qui sont deux des plus grands défis des systèmes RAG en production.

## 4.2 Appel de Fonctions et Utilisation d'Outils

L'appel de fonctions (*function calling*) est le mécanisme technique qui permet aux LLM d'interagir avec le monde extérieur et d'exécuter des actions concrètes. C'est le pont entre le raisonnement abstrait et l'action.

### L'API pour l'Action

L'appel de fonctions permet aux développeurs de définir un ensemble d'outils (par exemple, une fonction de requête de base de données, une API de recherche web, une calculatrice) que le LLM peut choisir d'invoquer pour répondre à une requête.<sup>29</sup> Il est crucial de comprendre que le modèle n'exécute pas la fonction lui-même. Au lieu de cela, il génère un objet JSON structuré contenant le nom de la fonction à appeler et les arguments à lui passer.<sup>29</sup> Le code du développeur est ensuite responsable de parser ce JSON, d'exécuter la

fonction correspondante avec les arguments fournis.

## Boucler la Boucle

Le résultat de l'exécution de la fonction est ensuite renvoyé au modèle en tant que nouveau contexte. Le modèle peut alors utiliser ce résultat pour formuler une réponse finale à l'utilisateur ou pour décider de l'étape suivante de son plan.<sup>29</sup> Ce processus en plusieurs étapes (utilisateur -> LLM -> appel de fonction -> code -> LLM -> utilisateur) constitue la base des agents augmentés par des outils. Le *Cookbook* d'OpenAI fournit des exemples clairs et détaillés de cette boucle complète.<sup>29</sup>

## Le RAG comme un Outil parmi d'Autres

Dans ce paradigme agentique, le système de récupération RAG n'est plus le composant central de l'architecture, mais devient un outil parmi d'autres dans la boîte à outils de l'agent. Face à une requête, un agent peut utiliser son raisonnement (souvent via un CoT) pour décider de la meilleure stratégie :

- Répondre directement à partir de ses connaissances paramétriques si la question est générale.
- Appeler l'outil de récupération RAG (`retrieve_documents`) si la question nécessite des connaissances spécifiques contenues dans la base de connaissances.
- Appeler un autre outil, comme une API de recherche web pour des informations en temps réel, un interpréteur de code pour des calculs, ou toute autre fonction définie par le développeur.

L'appel de fonctions est le pivot architectural qui unifie le RAG et le CoT pour créer de véritables agents. Il fournit l'API pratique qui permet au raisonnement abstrait du CoT de se traduire en actions concrètes, le RAG étant l'une de ces actions possibles. Sans l'appel de fonctions, un modèle pourrait générer un plan CoT qui dit « Je dois consulter le dernier rapport financier », mais il n'aurait aucun mécanisme pour le faire *réellement*. Avec l'appel de fonctions, le récupérateur RAG devient un outil invocable, et le raisonnement CoT de l'agent peut désormais inclure des étapes comme `tool_code(retrieve_documents(query="résultats T4 2025"))`. Le développement d'API d'appel de fonctions robustes a été le catalyseur technologique clé qui a permis aux concepts théoriques des agents (combinant raisonnement et action) de devenir une réalité pratique pour les développeurs.

## Partie V : Synthèse et Recommandations Stratégiques pour l'Implémentation

Cette section finale propose un cadre de prise de décision stratégique pour les praticiens, allant au-delà des descriptions techniques pour offrir des conseils exploitables pour la conception et le déploiement de systèmes intelligents et robustes.

### 5.1 Le Spectre des Architectures RAG : du Simple à l'Agentique

Les choix architecturaux pour un système RAG peuvent être présentés comme un modèle de maturité, chaque niveau offrant un compromis différent entre la simplicité, le coût et la robustesse.

- **Niveau 1 : RAG Naïf.** C'est l'implémentation la plus simple, suivant le pipeline linéaire de base (récupérer puis générer).
  - **Avantages :** Rapide à prototyper, faible complexité, latence et coût les plus bas.
  - **Cas d'utilisation :** Idéal pour des tâches de questions-réponses simples sur des corpus de documents propres, bien structurés et homogènes, où les requêtes des utilisateurs sont généralement bien définies.
- **Niveau 2 : RAG Avancé.** Ce niveau intègre une ou plusieurs des techniques d'amélioration décrites précédemment, telles que la recherche hybride, le re-classement ou les transformations de requêtes.
  - **Avantages :** Amélioration significative de la pertinence et de la précision par rapport au RAG naïf.
  - **Cas d'utilisation :** Nécessaire pour gérer des requêtes plus complexes, des sources de données bruyantes ou hétérogènes, ou lorsque la précision factuelle est une exigence plus stricte.
- **Niveau 3 : RAG Agentique.** Ce niveau implémente une boucle de raisonnement et d'auto-correction, où l'agent peut évaluer les informations récupérées et adapter sa stratégie de manière dynamique.
  - **Avantages :** Robustesse et fiabilité maximales. Capacité à gérer l'ambiguïté, les échecs de récupération et les tâches en plusieurs étapes.
  - **Cas d'utilisation :** Essentiel pour les applications critiques où la fiabilité est primordiale, comme les assistants juridiques, médicaux ou financiers, ou pour les agents de recherche autonomes capables de décomposer des questions complexes.

## 5.2 Un Cadre Décisionnel pour la Conception de Systèmes

Le choix de la bonne architecture dépend d'une analyse minutieuse des exigences de l'application. Un praticien peut évaluer son cas d'utilisation selon trois axes principaux :

1. **Complexité de la Requête** : Les utilisateurs poseront-ils des questions factuelles simples (par exemple, « Quelle est l'adresse de notre bureau de Paris? ») ou des questions nécessitant un raisonnement en plusieurs étapes et la synthèse de plusieurs sources (par exemple, « Comparez les performances financières de nos trois principaux concurrents au cours des deux dernières années et identifiez les risques communs mentionnés dans leurs rapports annuels »)?
2. **Qualité des Données** : La base de connaissances est-elle composée de documents propres, structurés et fiables (par exemple, une documentation interne bien entretenue) ou d'un corpus hétérogène et potentiellement bruyant (par exemple, des tickets de support client, des e-mails et des pages web externes)?
3. **Exigences de Fiabilité** : Le système peut-il se permettre une erreur occasionnelle (par exemple, un chatbot créatif) ou doit-il garantir une précision factuelle quasi parfaite (par exemple, un système fournissant des conseils médicaux ou financiers)?

Ces axes peuvent être utilisés pour guider le choix des techniques appropriées, comme illustré dans le tableau ci-dessous.

## 5.3 L'Avenir de l'IA à Forte Intensité de Connaissances

Le domaine évolue rapidement, mais plusieurs tendances clés se dessinent et façonneront l'avenir des systèmes RAG et agentiques.

- **La Centralité de la Qualité des Données** : La performance ultime de tout système RAG est limitée par la qualité des données auxquelles il a accès. L'adage « garbage in, garbage out » reste plus vrai que jamais. L'avenir verra une attention accrue portée aux pipelines de données en amont : nettoyage automatisé, analyse de documents sophistiquée (comme avec LlamaParse <sup>14</sup>) et construction de graphes de connaissances pour représenter les relations structurées au sein des données.<sup>30</sup>
- **L'Estompement des Frontières entre Récupération et Raisonnement** : À mesure que les modèles deviennent plus capables, la distinction entre la récupération d'informations et le raisonnement sur ces informations continuera de s'estomper. Les systèmes effectueront davantage de raisonnement *avant* la récupération (comme dans HyDE) pour

mieux comprendre l'intention, et une synthèse plus sophistiquée *après* la récupération pour tisser des liens entre plusieurs sources.

- **La Robustesse et la Sécurité comme Préoccupations de Premier Ordre :** À mesure que les systèmes RAG se généralisent, la recherche se concentrera de plus en plus sur la résolution de leurs vulnérabilités. Cela inclut la robustesse contre les données bruyantes ou contradictoires, la défense contre les entrées adverses conçues pour tromper le système (comme les attaques par empoisonnement de données <sup>31</sup>), et la garantie de la confidentialité des données lors de la récupération.<sup>31</sup>

| Archétype de Cas d'Utilisation           | Architecture Recommandée                    | Stratégie de Prompt Clé                                                      | Techniques RAG Fondamentales                                                          | Capacités Agentiques (si applicable)                                                   | Compromis Principaux                                                                     |
|------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Q&amp;R sur Documentation Interne</b> | Niveau 1 : RAG Naïf / Niveau 2 : RAG Avancé | Instructions claires et directes, utilisation d'exemples pour le formatage.  | Découpage sémantique, Filtrage par métadonnées.                                       | Aucune ou minimale.                                                                    | Simplicité et rapidité contre une gestion limitée de l'ambiguïté.                        |
| <b>Chatbot de Support Client</b>         | Niveau 2 : RAG Avancé                       | Persona (agent de support), instructions positives, décomposition de tâches. | Recherche hybride, Récupération Parent-Enfant.                                        | Aucune ou minimale.                                                                    | Meilleure pertinence pour un coût et une latence modérés.                                |
| <b>Assistant de Recherche</b>            | Niveau 3 : RAG Agentique                    | CoT Zero-Shot avec instruction de synthétiser plusieurs sources.             | Recherche hybride, Génération de requêtes multiples, Re-classement par Cross-Encoder. | Évaluation des documents, réécriture de la requête, utilisation d'outils (par exemple, | Latence et coût plus élevés pour une précision et une exhaustivité nettement supérieures |

|                                   |                                |                                                                    |                                                                              |                                                                                                                   |                                                                            |
|-----------------------------------|--------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
|                                   |                                |                                                                    |                                                                              | recherche web).                                                                                                   | .                                                                          |
| <b>Agent d'Analyse Financière</b> | Niveau 3 :<br>RAG<br>Agentique | CoT avec sortie structurée (JSON), règles et contraintes strictes. | Recherche hybride, Filtrage par métadonnées (dates, sources), Re-classement. | Évaluation de la pertinence, utilisation d'outils (calculatrice, interpréteur de code, API de données de marché). | Complexité maximale pour une fiabilité et une capacité d'action maximales. |

## Sources des citations

1. Best Practices for Prompting AI - Technology Help - Lafayette College, consulté le octobre 24, 2025, <https://help.lafayette.edu/best-practices-for-prompting-ai/>
2. Best practices for prompt engineering with the OpenAI API | OpenAI ..., consulté le octobre 24, 2025, <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>
3. Prompt Engineering for AI Guide | Google Cloud, consulté le octobre 24, 2025, <https://cloud.google.com/discover/what-is-prompt-engineering>
4. Anthropic just revealed their internal prompt engineering template - here's how to 10x your Claude results : r/PromptEngineering - Reddit, consulté le octobre 24, 2025, [https://www.reddit.com/r/PromptEngineering/comments/1n08dpp/anthropic\\_just\\_revealed\\_their\\_internal\\_prompt/](https://www.reddit.com/r/PromptEngineering/comments/1n08dpp/anthropic_just_revealed_their_internal_prompt/)
5. Google just released a 68-page guide on prompt engineering. Here are the most interesting takeaways - Reddit, consulté le octobre 24, 2025, [https://www.reddit.com/r/ChatGPTPromptGenius/comments/1kpvvvl/google\\_just\\_released\\_a\\_68page\\_guide\\_on\\_prompt/](https://www.reddit.com/r/ChatGPTPromptGenius/comments/1kpvvvl/google_just_released_a_68page_guide_on_prompt/)
6. Effective context engineering for AI agents \ Anthropic, consulté le octobre 24, 2025, <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
7. Prompt engineering techniques - Azure OpenAI | Microsoft Learn, consulté le octobre 24, 2025, <https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/prompt-engineering>

8. Prompt engineering for GitHub Copilot Chat, consulté le octobre 24, 2025, <https://docs.github.com/en/copilot/concepts/prompting/prompt-engineering>
9. Retrieval augmented generation (RAG) | 🦜 LangChain, consulté le octobre 24, 2025, <https://python.langchain.com/docs/concepts/rag/>
10. Advanced RAG Techniques | StackAI, consulté le octobre 24, 2025, <https://www.stack-ai.com/blog/advanced-rag-techniques>
11. LangChain RAG Implementation: Complete Tutorial with Examples - Latenode, consulté le octobre 24, 2025, <https://latenode.com/blog/langchain-rag-implementation-complete-tutorial-with-examples>
12. Simple RAG for GitHub issues using Hugging Face Zephyr and LangChain, consulté le octobre 24, 2025, [https://huggingface.co/learn/cookbook/rag\\_zephyr\\_langchain](https://huggingface.co/learn/cookbook/rag_zephyr_langchain)
13. RAG with LangChain - Docling, consulté le octobre 24, 2025, [https://docling-project.github.io/docling/examples/rag\\_langchain/](https://docling-project.github.io/docling/examples/rag_langchain/)
14. Welcome to LlamaIndex ! | LlamaIndex Python Documentation, consulté le octobre 24, 2025, <https://developers.llamaindex.ai/python/framework/>
15. RAG techniques: From naive to advanced - Weights & Biases - Wandb, consulté le octobre 24, 2025, <https://wandb.ai/site/articles/rag-techniques/>
16. Agentic RAG - GitHub Pages, consulté le octobre 24, 2025, [https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph\\_agentic\\_rag/](https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_agentic_rag/)
17. Llamaindex RAG Tutorial | IBM, consulté le octobre 24, 2025, <https://www.ibm.com/think/tutorials/llamaindex-rag>
18. RAG - Hugging Face, consulté le octobre 24, 2025, [https://huggingface.co/docs/transformers/model\\_doc/rag](https://huggingface.co/docs/transformers/model_doc/rag)
19. Retrieval Augmented Generation with Huggingface Transformers and Ray | Anyscale, consulté le octobre 24, 2025, <https://www.anyscale.com/blog/retrieval-augmented-generation-with-huggingface-transformers-and-ray>
20. RAG Evaluation - Hugging Face Open-Source AI Cookbook, consulté le octobre 24, 2025, [https://huggingface.co/learn/cookbook/rag\\_evaluation](https://huggingface.co/learn/cookbook/rag_evaluation)
21. Navigate through Enigmatic Labyrinth A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/html/2309.15402v3>
22. Revisiting Chain-of-Thought Prompting: Zero-shot Can Be Stronger than Few-shot - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/html/2506.14641v1>
23. dair-ai/Prompt-Engineering-Guide: Guides, papers, lessons ... - GitHub, consulté le octobre 24, 2025, <https://github.com/dair-ai/Prompt-Engineering-Guide>
24. [2308.10792] Instruction Tuning for Large Language Models: A Survey - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/abs/2308.10792>
25. Instruction Tuning for Large Language Models: A Survey - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/html/2308.10792v9>
26. Instruction Tuning for Large Language Models: A Survey - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/html/2308.10792v5>
27. [2410.10926] Federated Data-Efficient Instruction Tuning for Large Language



- Models - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/abs/2410.10926>
28. [2501.09136] Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG - arXiv, consulté le octobre 24, 2025, <https://arxiv.org/abs/2501.09136>
29. How to call functions with chat models - OpenAI Cookbook, consulté le octobre 24, 2025, [https://cookbook.openai.com/examples/how\\_to\\_call\\_functions\\_with\\_chat\\_models](https://cookbook.openai.com/examples/how_to_call_functions_with_chat_models)
30. Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers - ResearchGate, consulté le octobre 24, 2025, [https://www.researchgate.net/publication/392335133\\_Retrieval-Augmented\\_Generation\\_A\\_Comprehensive\\_Survey\\_of\\_Architectures\\_Enhancements\\_and\\_Robustness\\_Frontiers](https://www.researchgate.net/publication/392335133_Retrieval-Augmented_Generation_A_Comprehensive_Survey_of_Architectures_Enhancements_and_Robustness_Frontiers)
31. arxiv.org, consulté le octobre 24, 2025, <https://arxiv.org/html/2506.00054v1>